ETTCS801

Cryptography and Network Security

Securing Student Records: Risk Assessment, File Encryption, Integrity Verification and Firewall Filtering

Integrated Situation – Project Report

Student ID: *4202670008*
Lecturer: *TUMWINE Isaac*

Date: September 29, 2026
Repository: https://github.com/joymireillewright-sketch/cryptography-and-network-security-

All source code, test evidence and documentation referred to in this report are available in the public GitHub repository listed above.

Contents

1 Introduction **2**
1.1 Objectives . . . 2
1.2 Repository Structure . . . 2

2 Risk Assessment **2**
2.1 Assets . . . 2
2.2 Vulnerabilities and Consequences . . . 2
2.3 Risk Ranking . . . 2
2.4 Recommended Controls . . . 3

3 Encryption and Integrity Tool **4**
3.1 Design Overview . . . 4
3.2 Commands . . . 4
3.3 Key Management . . . 4
3.4 Encryption and Decryption . . . 4
3.5 SHA-256 Integrity Verification . . . 5
3.6 Error Handling . . . 5
3.7 Usage Workflow . . . 6

4 Network Traffic Filtering **6**
4.1 Network Configuration . . . 6
4.2 Firewall Policy . . . 6
4.3 Testing Method . . . 7
4.4 Test Results . . . 7

5 Discussion **7**
5.1 How the Components Address the Risks . . . 7
5.2 Limitations . . . 7
5.3 Possible Improvements . . . 8

6 Conclusion **8**

1 Introduction

This project addresses the security of a central student-records system. The scenario involves a records server accessed by authorised staff across campuses. The work is organised in three parts:

1. a **risk assessment** identifying assets, vulnerabilities, consequences, a risk ranking and recommended controls;
2. a **Python security tool** that encrypts and decrypts a sample student record, verifies file integrity with SHA-256, and handles errors gracefully;
3. a **network traffic filtering** design with a firewall policy and three documented tests.

1.1 Objectives

- Identify and rank the main security risks to the student-records system.
- Protect the confidentiality of a student record through symmetric encryption.
- Detect unauthorised modification of a file through cryptographic hashing.
- Restrict network access to the records server using a default-deny firewall policy.

1.2 Repository Structure

```
cryptography-network-security-exam/
|-- security_tool.py # encryption, decryption, hashing
|-- student_record.txt # sample plaintext record
|-- student_record.enc # encrypted record
|-- student_record_decrypted.txt# decrypted output
|-- risk_assessment.md # Section 1
|-- firewall_rules.sh # iptables equivalent (Linux)
|-- firewall_rules.txt # firewall policy and tests
|-- firewall_simulator.py # Windows-friendly rule simulator
|-- filter_tests.md # firewall test evidence
|-- README.md
|-- .gitignore
‘-- report/
    |-- report.tex
    ‘-- report.pdf
```

2 Risk Assessment

2.1 Assets

Three assets were identified as critical to the system.

2.2 Vulnerabilities and Consequences

2.3 Risk Ranking

Each risk was rated by likelihood and impact. All three carry a high impact because student records contain sensitive personal and academic data.

Asset	Description
Student records	Academic and personal information stored on the central server.
Student records server	The central server that stores and provides access to student records.
Network services	Services allowing authorised staff and campuses to access the records system.

Table 1: Identified assets.

Vulnerability	Possible consequence
Weak staff passwords	Unauthorised users may gain access to accounts and student records.
Unencrypted file transfers	Student information may be intercepted during transmission.
Guest network access to the records server	Unauthorised users may access or attack the records server.

Table 2: Vulnerabilities and their consequences.

Risk	Likelihood	Impact	Reason
Weak staff passwords	High	High	Weak credentials are easier to compromise and may give access to sensitive records.
Unencrypted file transfers	Medium	High	Data sent without encryption may be exposed during transfer.
Guest access to records server	Medium	High	Unauthorised network users could attempt to reach the records server.

Table 3: Risk ranking (highest priority first).

Risk	Control
Weak staff passwords	Enforce strong password requirements and secure authentication practices.
Unencrypted file transfers	Use encrypted communication when transferring student records.
Guest network access	Apply firewall rules and network segmentation to block guest access to the records server.

Table 4: Recommended controls.

2.4 Recommended Controls

The encryption tool (Section 3) addresses the confidentiality of stored and transferred records, while the firewall (Section 4) implements the network segmentation control.

3 Encryption and Integrity Tool

3.1 Design Overview

The tool `security_tool.py` is a command-line program written in Python 3. It uses the `cryptography` library's **Fernet** scheme for encryption and Python's standard `hashlib` module for SHA-256 hashing.

Fernet provides authenticated symmetric encryption: it combines AES-128 in CBC mode for confidentiality with an HMAC-SHA256 tag for authenticity. A modified or corrupted ciphertext, or a wrong key, is therefore detected at decryption time and raises an `InvalidToken` exception rather than returning garbage data.

3.2 Commands

Command	Purpose
`python security_tool.py key`	Generate the Fernet key outside the repository.
`python security_tool.py encrypt`	Encrypt `student_record.txt` to `student_record.enc`.
`python security_tool.py decrypt`	Decrypt the file and compare it with the original.
`python security_tool.py hash`	Compute and save the SHA-256 baseline.
`python security_tool.py verify`	Compare the current SHA-256 with the saved baseline.
`python security_tool.py help`	Display usage information.

Table 5: Command-line interface.

3.3 Key Management

A key stored next to the data it protects offers little security. The tool therefore saves `encryption.key` in the **parent directory** of the project, outside the Git repository, so that it cannot be committed by accident. Key generation refuses to overwrite an existing key, preventing accidental loss of access to previously encrypted data.

Listing 1: Key location and generation (excerpt).

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PARENT_DIR = os.path.dirname(BASE_DIR)
KEY_FILE = os.path.join(PARENT_DIR, "encryption.key")

def generate_key():
    if os.path.exists(KEY_FILE):
        print(f"Key already exists: {KEY_FILE}")
        print("No new key was generated.")
        return
    key = Fernet.generate_key()
    with open(KEY_FILE, "wb") as file:
        file.write(key)
```

3.4 Encryption and Decryption

Encryption reads the plaintext record in binary mode, encrypts it with the Fernet cipher and writes the result to `student_record.enc`. Decryption reverses the process, writes `student_record_decrypted.t` and then compares the decrypted bytes with the original file, reporting whether verification succeeded.

Listing 2: Decryption with error handling (excerpt).

```
try:
    decrypted = cipher.decrypt(encrypted)
except InvalidToken as error:
    raise ValueError(
        "Decryption failed: invalid key or corrupted encrypted data."
    ) from error

# ... write decrypted file, read original ...
if decrypted == original:
    print("Decryption successful.")
    print("Verification successful: decrypted content matches the original.")
else:
    print("Decryption completed, but verification FAILED.")
```

3.5 SHA-256 Integrity Verification

The `hash` command computes the SHA-256 digest of the record and stores it as a baseline. The `verify` command recomputes the digest and compares it to the stored value. Any change to the file, even a single bit, produces a completely different digest, so tampering is detected. The file is read in 4096-byte chunks, so the function works for large files without loading them entirely into memory.

Listing 3: SHA-256 of a file.

```
def sha256_file(filename):
    if not os.path.exists(filename):
        raise FileNotFoundError(f"File not found: {filename}")
    digest = hashlib.sha256()
    with open(filename, "rb") as file:
        for chunk in iter(lambda: file.read(4096), b""):
            digest.update(chunk)
    return digest.hexdigest()
```

3.6 Error Handling

The tool anticipates the following failure conditions and reports each with a clear message and a non-zero exit code instead of a stack trace.

Condition	Behaviour
Missing key file	Error message with the command needed to generate a key.
Missing input or encrypted file	`FileNotFoundError` caught and reported.
Wrong key or corrupted ciphertext	`InvalidToken` converted to a readable error.
Invalid key contents	`ValueError` reported as “Invalid encryption key”.
Missing hash baseline	Prompt to run the `hash` command first.
Unknown or missing command	Error message followed by usage help.
Key already exists	Existing key preserved; no new key generated.

Table 6: Error conditions handled by the tool.

3.7 Usage Workflow

```
python security_tool.py key # once: create key outside repo
python security_tool.py hash # save SHA-256 baseline
python security_tool.py encrypt # student_record.txt -> .enc
python security_tool.py decrypt # .enc -> decrypted file + comparison
python security_tool.py verify # integrity check against baseline
```

4 Network Traffic Filtering

4.1 Network Configuration

Parameter	Value
Records server / test host	`192.168.1.70`
Local / staff network	`192.168.1.0/24`
Default gateway	`192.168.1.254`
Simulated guest network	`192.168.2.0/24`
Service	Student Records Application
Protocol	TCP
Service port	`8443`

Table 7: Network parameters used for the firewall design and tests.

4.2 Firewall Policy

The design follows the *default deny* principle: everything not explicitly permitted is dropped. This directly implements the "firewall rules and network segmentation" control from the risk assessment.

Rule	Description
1	Allow established and related connections.
2	**DROP** all traffic from the guest network `192.168.2.0/24` to the records server `192.168.1.70`.
3	**ACCEPT** TCP traffic from the staff network `192.168.1.0/24` to `192.168.1.70` on port `8443`.
4	**DROP** TCP traffic to port `8443` from sources outside the authorised staff network.
5	**DROP** all other inbound traffic (default policy).

Table 8: Firewall rules (default policy: DROP).

Rule 2 is placed before the ACCEPT rule so that guest traffic is dropped explicitly and cannot be matched by a broader permit rule. The Linux `iptables` equivalent of this policy is provided in `firewall_rules.sh`.

4.3 Testing Method

The tests were run with `firewall_simulator.py` on Windows 11. Because no separate guest network was available on the test computer, the guest network was *simulated*. The `iptables` script was written for Linux and was therefore documented but not executed on Windows 11.

4.4 Test Results

#	Scenario	Source	Destination:Port	Expected	Actual	Status
1	Permitted staff	`192.168.1.50`	`192.168.1.70:8443`	ACCEPT	ACCEPT	PASS
2	Blocked guest	`192.168.2.75`	`192.168.1.70:8443`	DROP	DROP	PASS
3	Blocked external	`203.0.113.50`	`192.168.1.70:8443`	DROP	DROP	PASS

Table 9: Firewall test results (TCP, port 8443).

All three tests passed: the staff workstation reached the records service, while both the guest host and the external host (from the documentation range `203.0.113.0/24`) were dropped. The tests can be reproduced with:

```
python firewall_simulator.py
```

5 Discussion

5.1 How the Components Address the Risks

Risk	Addressed by
Unencrypted file transfers	Fernet encryption of records before storage or transfer.
Guest access to server	Default-deny firewall with explicit guest and external DROP rules.
Tampering with records	SHA-256 baseline and verification.
Weak staff passwords	Recommended policy; not implemented in code (organisational control).

Table 10: Mapping of risks to implemented or recommended controls.

5.2 Limitations

- The firewall tests used a simulator on Windows 11 and a simulated guest network; results confirm the rule logic but not behaviour on real hardware or a live `iptables` deployment.
- Fernet is a symmetric scheme, so secure key distribution and storage remain the operator's responsibility. Storing the key one directory above the repository prevents accidental commits but is not a substitute for a hardware security module or a managed secrets store.
- The SHA-256 baseline is stored in a file that an attacker with write access could also modify. For stronger assurance the baseline should be protected separately or replaced by an HMAC or digital signature.

- The tool handles a single sample record; a production system would need access control, auditing and key rotation.

5.3 Possible Improvements

- Password-derived keys (e.g. PBKDF2 or scrypt) or an external key vault.
- Multi-factor authentication for staff accounts.
- TLS for all traffic to port 8443, so data is protected in transit as well as at rest.
- Logging of dropped packets and periodic automated integrity checks.

6 Conclusion

The project delivers a coherent security solution for the student-records scenario. The risk assessment identified weak passwords, unencrypted transfers and guest network exposure as the key threats. The Python tool protects record confidentiality with authenticated encryption and detects modification with SHA-256 verification, while handling errors safely. The default-deny firewall policy isolates the records server, and all three tests behaved as expected. Together, these measures reduce the highest-ranked risks, and the limitations identified above point to clear next steps for a production deployment.

References

[1] Python Cryptographic Authority, *Fernet (symmetric encryption)*, `cryptography` documentation.

[2] NIST, *FIPS PUB 180-4: Secure Hash Standard (SHS)*, 2015.

[3] Netfilter Project, *iptables* documentation.

[4] Project repository: https://github.com/nicolasmulumba000-create/cryptography-network-security-